

Week 2 · 개념 워크북 — 참조가 순서를 만든다

📌 참고

이 문서는 개념 파트(12분)입니다. 손으로 치는 실습은 별도 문서 **실습워크북** (같은 **lecture/** 폴더) 을 따라갑니다.
[개념] 은 개념 설명, [함정] 은 미리 피해야 할 것입니다.

이 문서를 덮을 때: ① 내가 순서를 적지 않았는데 순서가 지켜지는 이유를 그래프로 설명할 수 있다. ② `terraform.tfstate` 를 열었을 때 어느 칸이 무엇인지 짚을 수 있다.

! 꼭 기억

Week 1을 다시 설명하지 않습니다. IaC·선언형·provider·4대 명령·state가 왜 필요한지는 Week1 개념워크북의 2·3·4·8·11번입니다. 오늘은 그 위에 얹습니다.

오늘은 Week1이 남긴 세 개의 예고를 회수합니다.

Week1에서	그때 한 말	오늘
7번 TIP	" <code>bucket = aws_s3_bucket.lab.id</code> 한 줄이 순서를 만든다. 이걸 암묵적 의존성이라 하고 Week2 주제다"	4번
7번 NOTE	" ~ 대신 <code>-/+</code> 가 뜰 때가 있다. 언제 그런지는 뒤 주차에서"	12번
[함정] 콘솔 수정	"콘솔에서 손으로 고치면 다음 apply에서 사라진다"	11번

오늘 다룰 것

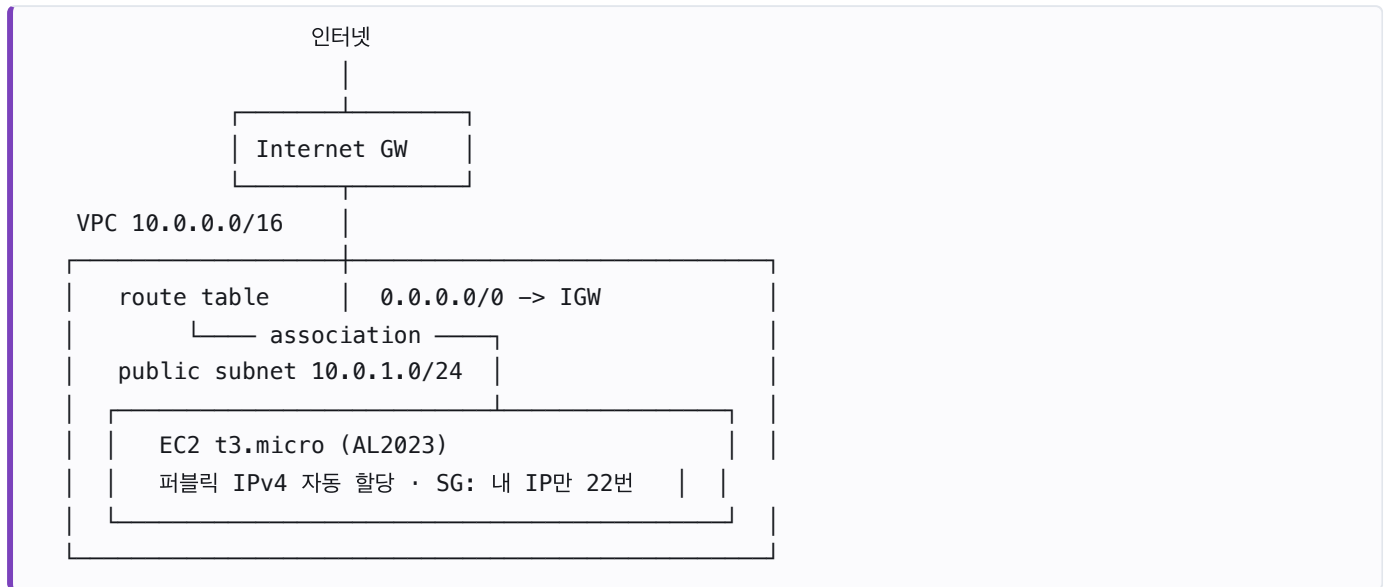
📌 참고

14개 섹션이지만 라이브에서는 1 · 2 · 4 · 7 · 9번만 짚습니다(12분). 나머지는 실습 중 막혔을 때 돌아와서 읽는 용도입니다. 그리고 라이브 12분은 실습 시간을 잡아먹지 않습니다 — `plan` 과 `apply` 가 도는 대기 시간에 얹습니다.

[개념] 1. 오늘의 그림 — 리소스는 7개, 화살표는 11개

핵심: 오늘 배우는 건 리소스 7개가 아니라 그 사이의 화살표 11개다.

Week1 1번에서 3계층 그림을 보며 "이 5개는 오늘 만들지 않습니다"라고 했습니다. 그중 네트워크와 VM을 오늘 진짜로 만듭니다.



리소스는 **7개**입니다. VPC 1 + subnet 1 + IGW 1 + route table 1 + association 1 + security group 1 + EC2 1.
 그런데 오늘의 주제는 이 7개가 아닙니다. **7개를 잇는 참조 11개**입니다. 이게 오늘 문서의 좌표계이니 표로 전부 잡아둡니다.

#	참조를 쓰는 곳	가리키는 대상	왜 필요한가
①	<code>aws_subnet.public</code> 의 <code>vpc_id</code>	<code>aws_vpc.main</code>	서브넷은 VPC 안에만 존재한다
②	<code>aws_subnet.public</code> 의 <code>availability_zone</code>	<code>data.aws_availability_zones.available</code>	쓸 수 있는 AZ를 AWS에 물어본다
③	<code>aws_internet_gateway.gw</code> 의 <code>vpc_id</code>	<code>aws_vpc.main</code>	IGW는 VPC에 붙는다
④	<code>aws_route_table.public</code> 의 <code>vpc_id</code>	<code>aws_vpc.main</code>	라우트 테이블도 VPC 소속
⑤	<code>aws_route_table.public</code> 의 <code>route.gateway_id</code>	<code>aws_internet_gateway.gw</code>	0.0.0.0/0의 출구가 IGW
⑥	<code>aws_route_table_association.public</code> 의 <code>subnet_id</code>	<code>aws_subnet.public</code>	어느 서브넷에 붙일지
⑦	<code>aws_route_table_association.public</code> 의 <code>route_table_id</code>	<code>aws_route_table.public</code>	어느 테이블을 붙일지
⑧	<code>aws_security_group.web</code> 의 <code>vpc_id</code>	<code>aws_vpc.main</code>	SG는 VPC 소속
⑨	<code>aws_instance.web</code> 의 <code>subnet_id</code>	<code>aws_subnet.public</code>	어느 서브넷에 뜯지
⑩	<code>aws_instance.web</code> 의 <code>vpc_security_group_ids</code>	<code>aws_security_group.web</code>	리스트라 [] 로 감싼다
⑪	<code>aws_instance.web</code> 의 <code>ami</code>	<code>data.aws_ami.al2023</code>	유일하게 <code>data.</code> 로 시작하는 참조

이 11개가 곧 그래프의 간선 11개입니다. 4번에서 이 표를 그래프로 세웁니다.

! 꼭 기억

오늘은 EC2에 SSH로 접속하지 않습니다. 키페어를 만들지 않으므로 접속할 수도 없습니다. 키를 코드로 만들면 비밀키가 tfstate에 평문으로 저장되고(8번), .pem 파일이 커밋 사고를 냅니다. 오늘의 성공 기준은 접속이 아니라 `instance_state = running` + 퍼블릭 IP 할당입니다.

[개념] 2. 리소스 주소와 참조 문법 — 세 조각

핵심: `aws_vpc.main.id` 는 타입·이름·속성 세 조각이고, 오늘의 문법은 이 세 조각이 전부다.

Week1 A-5에서 `aws_s3_bucket.lab` 이 리소스 주소라는 걸 배웠습니다. 오늘은 여기에 한 칸이 더 붙습니다.

<code>aws_subnet.public.id</code>	<code>data.aws_ami.al2023.id</code>
+- 속성	+- 속성
+----- 이름	+----- 이름
+----- 타입	+----- 타입
	+----- data 접두사

주소는 두 조각(`aws_subnet.public`), 참조는 세 조각(`aws_subnet.public.id`) 입니다. 주소는 "누구"이고, 참조는 "누구의 무엇"입니다.

그리고 여기서 초보가 가장 많이 헷갈리는 게 이름이 세 층에 있다는 사실입니다.

층	예	누가 정하나	AWS에 올라가나
코드 안 이름	<code>main</code> , <code>public</code> , <code>web</code>	내가	아니오
리소스 주소	<code>aws_subnet.public</code>	타입 + 이름 조합	아니오 (state-plan-graph에만 나옴)
AWS 실제 ID	<code>subnet-0a1b2c3d4e5f</code>	AWS가 apply 때	예 — <code>.id</code> 로 받아오는 그것

읽는 규칙은 하나입니다. 등호 왼쪽은 인자 이름(provider가 정함), 오른쪽은 참조(내가 씀).

```
vpc_id = aws_vpc.main.id
# ^           ^
# |           +-- 오른쪽: 내가 붙인 주소 + 속성
# +----- 왼쪽: aws_subnet 이 요구하는 인자 이름
```

세부 문법 셋만 짚습니다.

- 대괄호: `vpc_security_group_ids = [aws_security_group.web.id]` — 인자가 리스트입니다. SG는 한 인스턴스에 여러 개 붙을 수 있어서 그렇습니다. 하나만 붙여도 `[ ]` 가 필요합니다.
- `${}` 는 문자열에 섞을 때만: `tags = { Name = "${var.project_name}-vpc" }` 처럼 문자열 안에 넣을 때만 씁니다. `vpc_id = aws_vpc.main.id` 는 값 하나를 그대로 쓰는 것이므로 `"${...}"` 로 감싸지 않습니다.
- 리스트의 원소 꺼내기: `data.aws_availability_zones.available.names[0]` — `names` 가 `["ap-northeast-2a", "2b", ...]` 라서 `[0]` 으로 첫 번째를 고릅니다.

⚠ 주의

코드 안 이름을 바꾸는 건 공짜가 아닙니다. `aws_vpc.main` 을 `aws_vpc.primary` 로 바꾸면 AWS에는 아무것도 안 알려주지만, 주소가 바뀌어서 Terraform은 "`main` 은 없어졌고 `primary` 가 새로 생겼다"고 판단합니다. plan에 `-/+` 가 뜨고 VPC가 통째로 재생성됩니다. 이유는 9번에 있습니다 — state가 리소스를 주소로 기억하기 때문입니다. (주소만 옮기는 `moved` 블록은 뒤 주차에서)

[개념] 3. (known after apply) 의 정체 — 아직 없는 값을 가리키기

핵심: 참조 대상이 "AWS가 만들어줘야 아는 값"이라서 순서가 생긴다.

Week1 A-8에서 (known after apply) 를 보고 "오류가 아닙니다"까지만 말했습니다. 오늘은 그게 의존성의 원인입니다.

`aws_subnet.public` 에 적은 `vpc_id = aws_vpc.main.id` 를 생각해봅시다. `aws_vpc.main.id` 의 실제 값은 `vpc-0f9e8d7c6b5a` 처럼 생겼는데, 이 값은 AWS가 VPC를 실제로 만들어준 뒤에야 존재합니다.

```
plan 시점 : vpc_id = (known after apply)      <- 아직 세상에 없는 값
apply 이후 : vpc_id = "vpc-0f9e8d7c6b5a"      <- 이제 값이 있다
```

그래서 논리가 이렇게 이어집니다.

1. 서브넷을 만들려면 `vpc_id` 가 필요하다
2. 그 값은 VPC를 만들어야 정해진다
3. 그러므로 VPC가 먼저다

"값이 나중에 정해진다"가 곧 "순서가 있다"입니다. plan에 (known after apply) 가 많이 보인다면, 그건 그만큼 화살표가 많다는 증거입니다. 반대로 `cidr_block = "10.0.1.0/24"` 처럼 내가 직접 적은 값은 plan에서 이미 확정돼 있습니다.

[개념] 4. 암묵적 의존성이 그래프가 되는 원리 ★ (이 파트의 핵심)

핵심: 참조 하나가 간선 하나다. Terraform은 그 간선들로 방향 그래프를 만들고, 위상 순서로 실행한다.

Week1은 "순서가 지켜진다"는 현상까지 말했습니다. 오늘은 그 기계적 절차로 내려갑니다.

[1] 코드 읽기	[2] 간선 추출	[3] 그래프	[4] 실행 순서
.tf 전체 파싱	-> 참조 1개 = 간선 1개	-> 방향 그래프	-> 레벨별 병렬 실행
	(1번 표의 11개)	(DAG)	(레벨 0~4)

📌 참고

용어 · DAG (Directed Acyclic Graph, 방향 비순환 그래프) — 화살표에 방향이 있고, 출발지로 돌아오는 고리가 없는 그래프. 고리가 있으면 시작점을 정할 수 없습니다(13번).

용어 · 위상 순서(topological order) — 모든 화살표가 같은 방향을 향하도록 줄을 세우는 것. "A가 B를 참조하면 B가 A보다 먼저"를 전부 만족하는 순서.

1번 표의 화살표 11개를 세우면 레벨 5단이 나옵니다. 오늘 문서에서 가장 중요한 표입니다.

레벨	리소스	왜 이 레벨인가
0	<code>data.aws_ami.al2023</code> , <code>data.aws_availability_zones.available</code>	아무것도 참조하지 않음. plan 단계에 이미 읽힌다
1	<code>aws_vpc.main</code>	아무것도 참조하지 않는 유일한 리소스. 전체의 병목
2	<code>aws_subnet.public</code> , <code>aws_internet_gateway.gw</code> , <code>aws_security_group.web</code>	VPC만 기다리면 된다 (①③⑧). subnet은 ②로 데이터 소스도 참조하지만 그건 레벨 0이라 이미 준비됨 → 3개 병렬
3	<code>aws_route_table.public</code> , <code>aws_instance.web</code>	RT는 VPC+IGW(④⑤), EC2는 subnet+SG+AMI(⑨⑩⑪) → 2개 병렬
4	<code>aws_route_table_association.public</code>	subnet(⑥) + route table(⑦) 둘 다 필요 → 마지막

💡 팁

가장 늦게 시작하는 건 EC2가 아니라 association입니다. 인자가 참조 두 개뿐인 가장 단순한 리소스가 그래프에서는 가장 마지막입니다. 그런데 시계로 재면 EC2가 가장 늦게 끝납니다(생성에 30~60초). 그래프 순서와 완료 시간 순서는 다릅니다. 실습 B-3에서 로그로 확인합니다.

그리고 여기가 오늘의 진짜 배울 점입니다.

! 꼭 기억

`aws_instance.web` 은 `aws_route_table_association` 에 의존하지 않습니다. 1번 표를 다시 보세요 — EC2가 참조하는 건 subnet·SG·AMI뿐입니다. 즉 라우트가 아직 안 붙은 상태에서 인스턴스가 먼저 만들어질 수 있습니다.

오늘은 문제가 안 됩니다. 인스턴스가 부팅해서 인터넷으로 뭘 하지 않으니까요. 하지만 `user_data` 에 `dnf install nginx` 를 넣는 순간 인터넷이 아직 없어서 실패하는 경쟁 상태가 생깁니다. 코드에 안 적힌 의존이 존재하는 것이고, 이게 6번 `depends_on` 이 필요한 이유입니다.

"참조가 곧 순서"의 한계가 바로 여기입니다. 참조가 없으면 순서도 없습니다.

destroy는 이 그래프의 화살표를 전부 뒤집은 것입니다. Week1 C-1에서 본 "만들 때의 역순"이 여기서 일반화됩니다.

```

apply   : 레벨 0 → 1 → 2 → 3 → 4      (뿌리부터)
destroy : 화살표를 뒤집은 그래프에서 다시 계산 (잎사귀부터)

```

```

[1] aws_instance.web | aws_route_table_association.public
[2] aws_security_group.web | aws_subnet.public | aws_route_table.public
[3] aws_internet_gateway.gw
[4] aws_vpc.main

```

! 꼭 기억

삭제 순서는 apply 레벨 숫자를 그대로 거꾸로 읽는 것이 아닙니다. apply에서 레벨 3이던 `instance` 와 레벨 4이던 `association` 이 삭제에서는 같은 1단계입니다. 기준이 "내가 무엇을 기다리나"에서 "나를 기다리는 것이 아직 남아 있나" 로 바뀌기 때문입니다. 아무도 참조하지 않는 것(잎사귀)부터 지웁니다.

이 그래프를 눈으로 보는 명령이 `terraform graph` 입니다. DOT라는 텍스트 형식을 출력하는데, 그림 프로그램 없이도 화살표만 뿔아 읽을 수 있습니다(실습 A-7).

[개념] 5. 그래프가 만드는 병렬 실행 — 왜 subnet·igw·sg가 동시에 뜨나

핵심: 같은 레벨에 있는 리소스는 동시에 처리된다. 다만 동시 실행 수의 상한은 그래프의 폭이다.

Terraform은 기본적으로 최대 10개를 동시에 처리합니다(`-parallelism=10`).

```
레벨 2 : subnet | igw | sg      <- 서로를 참조하지 않으므로 동시에 시작
레벨 3 : route table | instance <- 마찬가지로
```

참고

`-parallelism` 은 의존성을 무시하지 않습니다. "동시에 최대 몇 개까지"만 정합니다. 오늘 구성에서 한 레벨의 최대 폭은 3이므로, 이 값을 100으로 올려도 1초도 빨라지지 않습니다. 병렬성의 상한은 옵션이 아니라 내 코드의 그래프 폭이 정합니다.

그리고 기본값을 임의로 올리면 AWS API 호출 제한(rate limit)에 걸립니다. 건드리지 마세요.

apply 로그에서 이걸 눈으로 확인할 수 있습니다. 같은 초에 `Creating...` 이 뜬 것들이 같은 레벨입니다.

```
aws_subnet.public: Creating...      <- 같은 순간
aws_internet_gateway.gw: Creating... <- 같은 순간
aws_security_group.web: Creating...  <- 같은 순간
```

전체 소요 시간은 EC2가 지배합니다. provider가 인스턴스가 `running` 이 될 때까지 기다리므로 보통 30~60초 걸리고, 나머지 6개는 합쳐도 5~10초입니다.

[개념] 6. `depends_on` — 참조가 없는데 순서가 필요할 때

핵심: `depends_on` 은 코드에 드러나지 않는 의존만을 위한 비상구이고, 참조로 되는 일에 쓰면 안 된다.

먼저 안 쓰는 이유부터 봅시다. `vpc_id = aws_vpc.main.id` 라고 적은 자리에 `depends_on = [aws_vpc.main]` 을 또 쓰면 그건 중복입니다. 그리고 중복은 언젠가 거짓말이 됩니다 — 참조는 고쳤는데 `depends_on` 은 안 고치는 일이 생깁니다.

그래서 규칙은 "참조로 표현할 수 있으면 참조로" 입니다.

그럼 언제 필요한가? 코드에 안 드러나는 의존이 있을 때입니다.

상황	왜 참조로 안 되나
EC2의 <code>user_data</code> 가 부팅 중 인터넷에서 패키지를 받는다	코드 어디에도 IGW·라우트를 참조하는 인자가 없다 (4번의 그 경쟁 상태)
IAM 정책을 role에 붙인 다음 EC2가 떠야 한다	정책이 붙기 전에 뜨면 권한 없이 부팅된다
S3 버킷에 객체를 올린 뒤 그 객체를 앱이 읽는다	런타임 의존이라 인자로 표현되지 않는다

문법에서 한 가지만 주의합니다.

```
depends_on = [aws_internet_gateway.gw] # 주소 (0)
depends_on = [aws_internet_gateway.gw.id] # 속성 (X)
```

depends_on 은 속성이 아니라 주소 목록입니다. "무엇의 값이 필요하다"가 아니라 "누구보다 나중"이라는 뜻이니깐요.

⚠ 주의

depends_on 으로 13번의 cycle을 우회할 수는 없습니다. 오히려 없던 고리를 만들어 상황을 악화시킵니다.

[개념] 7. 데이터 소스 심화 — AMI를 하드코딩하지 않는 이유

핵심: **data** 는 AWS에 질문하는 블록이고, 질문이 헛거우면 답이 매주 달라진다.

Week1 A-4의 `data "aws_caller_identity" "current" {}` 는 인자가 0개였습니다. 물어볼 게 "내가 누구냐" 하나뿐이었으니까요. 오늘은 질문에 조건을 붙입니다.

📌 참고

용어 · AMI (Amazon Machine Image) — EC2가 부팅할 때 쓰는 디스크 이미지. OS와 초기 소프트웨어가 들어 있습니다. `ami-04bb4ddfc0e51bb5e` 처럼 생긴 ID로 지정합니다.

```
data "aws_ami" "al2023" {
  most_recent = true           # 조건에 맞는 것 중 가장 최근 것
  owners      = ["amazon"]    # 누가 만든 이미지인가
  filter {                     # 어떤 조건으로 걸러낼 것인가
    name      = "name"
    values    = ["al2023-ami-2023.*-x86_64"]
  }
  # architecture / virtualization-type / root-device-type / state 필터가 더 붙습니다
}
```

왜 `ami = "ami-0abc..."` 로 적지 않는가?

하드코딩하면	결과
서울에서 쓴 코드를 도쿄로 복사	<code>InvalidAMIID.NotFound</code> — AMI ID는 리전마다 다릅니다
3개월 뒤 그 이미지가 폐기(deprecated)됨	apply 실패, 또는 보안 패치가 안 된 이미지로 계속 기동
arm64용 AMI에 <code>t3.micro</code> (x86_64)	아키텍처 불일치로 부팅 불가

그런데 질문이 헛거우면 더 무섭습니다. 이 실습 코드의 원래 필터는 `["al2023-ami-*-*x86_64"]` 였습니다. 실제로 서울 리전에서 조회해보면 이 조건에 이런 것들이 다 걸립니다.

걸리는 AMI 이름	루트 볼륨	문제
<code>al2023-ami-ecs-neuron-hvm-...</code>	30 GiB	ECS 전용 이미지. <code>most_recent</code> 가 실제로 이걸 고릅니다
<code>al2023-ami-ecs-hvm-...</code>	30 GiB	ECS 전용
<code>al2023-ami-minimal-...</code>	2 GiB	SSM Agent·awscli가 빠진 최소 이미지
<code>al2023-ami-2023.12...-x86_64</code>	8 GiB	우리가 원하는 것

30 GiB짜리를 띄우면 EBS 비용이 3.75배가 되고 프리티어 EBS 30GB를 한 대로 다 씁니다. 그래서 이름 패턴을 `al2023-ami-2023.*` 로 시작까지 고정했습니다.

🚫 절대 금지

`owners = ["amazon"]` 은 방어선이 못 됩니다. ECS 전용 AMI를 발행하는 AWS 서비시팀 계정도 소유자 별칭이 `amazon` 입니다. 그래도 `owners` 는 반드시 남겨야 합니다 — 빼면 아무나 올린 커뮤니티 AMI가 후보에 들어옵니다.

🚫 절대 금지

`most_recent = true` 의 대가: 같은 코드가 다음 주엔 다른 AMI를 고릅니다. 그리고 `ami` 는 12번에서 볼 교체(-/+) 속성입니다. 즉 코드를 한 글자도 안 고쳤는데 다음 apply에서 EC2가 통째로 재생성될 수 있습니다. 실무에서 재현성이 필요하면 AMI ID를 변수로 고정하거나 SSM 파라미터로 버전을 못 박습니다.

AZ도 같은 이유로 물어봅니다.

```
data "aws_availability_zones" "available" { state = "available" }
# 사용: availability_zone = data.aws_availability_zones.available.names[0]
```

! 꼭 기억

`availability_zone = "ap-northeast-2a"` 를 하드코딩하면 안 되는 이유는 두 갈래입니다.

- AZ 이름은 계정마다 다른 물리 AZ에 매핑됩니다. 내 `2a` 와 옆 사람의 `2a` 는 다른 건물일 수 있고, 오래된 계정은 특정 AZ에 서브넷을 못 만들 수도 있습니다.
- AZ × 인스턴스 타입 조합이 있습니다. 서울 리전 실측으로 `t3.micro` 는 2a/2b/2c/2d 전부 가능하지만 `t2.micro` 는 2b·2d에 없습니다. `t2.micro` 로 바꾼 사람이 `2b` 를 쓰면 `Unsupported` 에러가 납니다.

[개념] 8. data 는 언제 읽히나 — plan 시점과 그 예외

핵심: `data` 는 plan에서 읽히지만, 참조 대상이 아직 없으면 apply로 밀린다.

Week1에서 두 가지를 봤습니다. ① plan 맨 위에 `Reading...` 두 줄이 먼저 뜬다 ② `state list` 에 데이터 소스도 나온다. 이 둘은 같은 원리입니다.

데이터 소스는 plan 단계에서 AWS에 질문하고, 그 답을 state에 `mode: "data"` 로 기록합니다. 그래서 오늘 plan에서 `ami` 와 `subnet_az` 는 (known after apply) 가 아니라 **확정된 값으로** 찍힙니다.

```
+ ami_id      = "ami-04bb4ddfc0e51bb5e"
+ subnet_az   = "ap-northeast-2a"
```

예외: 데이터 소스의 인자가 아직 만들어지지 않은 리소스를 참조하면(예: 방금 만들 VPC의 ID로 뭔가를 조회) plan 때 읽을 수 없어 **apply**로 밀립니다. 그러면 그 값을 쓰는 곳마다 (known after apply) 가 번집니다.

그리고 destroy할 때 데이터 소스는 **N destroyed** 숫자에 들어가지 않습니다. 만든 적이 없으니 지울 것도 없습니다.

[개념] 9. tfstate 내부 — JSON 7개 키와 dependencies 배열 ☆

핵심: tfstate는 리소스 배열 하나에 "주소 + provider + 속성 + 의존성"을 적어둔 장부다.

Week1 8번은 "장부가 있다"까지, B-2는 "평문이다"까지 말했습니다. 오늘은 칸을 하나씩 뜯습니다.

`terraform.tfstate` 의 최상위 키는 7개입니다.

키	값 예	뜻	내가 신경 쓰나
<code>version</code>	4	state 파일 포맷 버전. Terraform 버전이 아님	아니오
<code>terraform_version</code>	"1.15.8"	이 state를 마지막으로 쓴 Terraform	팀원 버전이 더 낮으면 에러
<code>serial</code>	3	state가 저장된 횟수. 저장마다 +1 (apply 횟수와 꼭 같지는 않다)	Week3 잠금(lock)의 열쇠
<code>lineage</code>	"a1b2c3d4-..."	이 state 파일의 혈통 UUID	다른 state와 섞이는 것을 막아줌
<code>outputs</code>	{ "vpc_id": {...} }	output 블록의 값	예
<code>resources</code>	[...]	본문. 관리 중인 모든 것	예
<code>check_results</code>	null	check 블록 결과 (오늘 안 씀)	아니오

`resources` 배열의 원소 하나를 뜯으면 이렇습니다.

필드	값 예	내 코드의 무엇
<code>mode</code>	"managed" / "data"	<code>resource</code> 블록 / <code>data</code> 블록
<code>type</code> · <code>name</code>	"aws_subnet" · "public"	주소의 두 조각 (2번)
<code>provider</code>	provider["registry.terraform.io/hashicorp/aws"]	Week1 A-2의 <code>source</code>
<code>instances[].attributes</code>	{ "id": "subnet-0a1b...", "vpc_id": "vpc-0f9e...", ... }	AWS가 돌려준 모든 속성 (평문)
<code>instances[].dependencies</code>	["aws_vpc.main", "data.aws_availability_zones.available"]	4번 그래프가 저장된 자리

! 꼭 기억

이 문서에서 가장 중요한 한 줄입니다. `dependencies` 배열은 4번에서 그린 그래프의 디스크 저장본입니다. 나는 그걸 적은 적이 없습니다 — 참조에서 자동으로 뽑혀 여기 적혔습니다. `aws_instance.web` 의 `dependencies` 를 열면 1번 표의 화살표 ⑨⑩⑪이 그대로 들어 있습니다.

! 꼭 기억

`destroy`가 역순으로 되는 이유가 여기 있습니다. `.tf` 파일을 전부 지워도 state의 `dependencies` 만 있으면 삭제 순서를 계산할 수 있습니다. 코드가 사라지면 참조도 사라지지만, 장부에 적힌 그래프는 남습니다.

📌 참고

`dependencies` 에는 직접 의존뿐 아니라 간접(전이) 의존까지 적히는 경우가 있습니다. 예를 들어 `association`의 `dependencies` 에 `aws_vpc.main` 이 함께 들어갈 수 있습니다. 그래프의 간선 수와 정확히 1:1은 아니라는 점만 알아두세요.

🚫 절대 금지

`attributes` 에는 우리가 쓴 것보다 훨씬 많은 값이 들어갑니다. AWS가 정해준 기본값까지 전부입니다. 그리고 `plan`이 ~ 인지 `-/+` 인지 계산하는 근거가 바로 이 값들입니다. 전부 평문입니다 — EC2의 `user_data` 도, DB 비밀번호도 마찬가지입니다. 오늘 `terraform show -json > state.json` 으로 뽑는 파일도 state와 같은 등급의 비밀입니다.

[개념] 10. state 명령군 — 장부를 읽는 여섯 가지 방법

핵심: state는 파일을 여는 게 아니라 명령으로 읽는다. 손으로 편집하는 순간 장부가 거짓말을 시작한다.

명령	보여주는 것	안전한가
<code>terraform state list</code>	주소 목록만	읽기
<code>terraform state show <주소></code>	그 하나의 모든 속성 (사람이 읽는 형식)	읽기
<code>terraform show</code>	state 전체 (사람이 읽는 형식)	읽기
<code>terraform show -json</code>	state 전체를 기계용 JSON으로	읽기
<code>terraform state pull</code>	장부 원본 JSON을 그대로 출력	읽기
<code>terraform output -json</code>	outputs만	읽기
<code>state mv</code> · <code>state rm</code> · <code>import</code> · <code>state push</code>	장부를 고침	위험 — 뒤 주차

Week1 B-2에서 앞의 셋은 이미 써봤습니다. 오늘 새로 쓰는 건 `-json` 과 `state pull` 입니다.

📌 참고

`terraform state pull` 은 로컬 state에서는 `cat terraform.tfstate` 와 거의 같습니다. 이 명령이 진짜 의미를 갖는 건 Week3 S3 원격 backend에서입니다 — 원격 장부를 내려받는 명령이죠. 짝인 `state push` 는 위험해서 오늘 쓰지 않습니다.

🚫 절대 금지

`terraform.tfstate` 를 에디터로 열어 고치지 마세요. `serial` · `lineage` 가 어긋나면 Terraform이 장부를 거부하거나, 더 나쁘게는 실제로 존재하는 리소스를 잊어버립니다. 잊힌 리소스는 아무도 지우지 않고 청구서로만 남습니다. 읽는 건 자유, 쓰는 건 명령으로만.

[개념] 11. drift와 refresh — 장부와 현실이 어긋날 때

핵심: 모든 `plan` 은 시작할 때 현실을 다시 읽는다(refresh). drift는 그때 발견된다.

Week1의 [함정] 콘솔에서 손으로 고치면 을 이제 메커니즘으로 봅니다.

📌 참고

용어 · drift(표류) — 코드·장부와 실제 클라우드가 어긋난 상태. 용어 · refresh — `plan` / `apply` 시작 시 state에 적힌 리소스를 AWS에 다시 조회해 현재 값을 확인하는 단계.

세 값(코드 / 장부 / 실제)의 조합으로 plan이 하는 말이 정해집니다.

code (.tf)	state (장부)	real (AWS)	plan이 하는 말
A	A	A	No changes.
A	A	B	drift. Objects have changed outside of Terraform + ~ 로 A로 되돌리겠다
B	A	A	정상 변경. ~ 로 B로 바꾸겠다
A	A	없음	+ 다시 만들겠다 (누가 콘솔에서 지웠다)
없음	A	A	- 지우겠다 (내가 코드에서 지웠다)
A	없음	A	Terraform이 모르는 리소스. + 로 만들려다 AlreadyExists 충돌

2행이 오늘 실습(B-5)에서 직접 만들 상황입니다. 그리고 화살표 방향이 핵심입니다.

```
~ tags = { ~ "Name" = "내가-콘솔에서-바꾼-값" -> "코드에-적힌-값" }
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
           Terraform은 이걸 '반영'하지 않고 '되돌립니다'
```

코드가 정답이고 AWS는 그 사본입니다. 그래서 IaC를 도입한 팀은 콘솔에서 손으로 고치는 걸 금지합니다 — 다음 apply에서 조용히 사라지니까요.

장부만 현실에 맞추고 싶을 때 쓰는 명령이 둘 있습니다.

명령	하는 일
<code>terraform plan -refresh-only</code>	AWS를 읽어 state와 다른 점만 보여줌. 아무것도 안 바꿈
<code>terraform apply -refresh-only</code>	실제 AWS 값을 state에 반영. 인프라는 안 바꿈

📌 참고

옛 명령 `terraform refresh` 는 **deprecated**입니다. 확인 프롬프트 없이 state를 즉시 덮어쓰기 때문입니다. `apply -refresh-only` 는 plan을 보여주고 `yes` 를 받습니다.

[개념] 12. ~ vs -/+ — 고칠 수 있는 속성과 못 고치는 속성

핵심: 어떤 속성은 제자리 수정이 불가능해서 리소스가 통째로 교체된다. plan의 그 한 줄을 못 읽으면 서버가 사라진다.

Week1 7번 마지막 NOTE(" -/+ 가 뜰 때가 있다. 언제인지는 뒤 주차에")를 여기서 회수합니다.

📌 참고

용어 · **force replacement (ForceNew)** — provider가 속성별로 "제자리 수정 가능/불가"를 정해줬습니다. AWS API에 그 필드를 바꾸는 수단이 없으면 방법은 지우고 다시 만드는 것뿐입니다.

plan 출력에서 읽을 곳은 딱 두 군데입니다.

Resource actions are indicated with the following symbols:

-/+ destroy and then create replacement

```
# aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
    ~ ami = "ami-04bb..." -> "ami-01e9..." # forces replacement    <- ①
    ...
}
```

Plan: 1 to add, 0 to change, 1 to destroy. <- ②

① **# forces replacement** 주석은 리소스 헤더가 아니라 원인이 된 속성 줄의 맨 끝에 붙습니다. 그 속성이 범인이라는 뜻입니다. ② 아랫줄 숫자가 **1 to change** 가 아니라 **1 to add, 1 to destroy** 입니다. 고치는 게 아니라 죽이고 새로 만드는 것이니까요.

오늘 만드는 리소스 기준으로 정리하면 이렇습니다.

리소스 · 속성	plan	실제로 무슨 일이
<code>aws_instance</code> · <code>tags</code>	~	무중단
<code>aws_instance</code> · <code>instance_type</code>	~	중지 → 변경 → 시작. 교체는 아니지만 다운타임이 있다
<code>aws_instance</code> · <code>ami</code>	-/+	인스턴스 교체. 디스크 데이터 소멸, 퍼블릭 IP 변경
<code>aws_instance</code> · <code>subnet_id</code>	-/+	교체
<code>aws_subnet</code> · <code>cidr_block</code> , <code>availability_zone</code>	-/+	교체 → 그 subnet을 참조한 것들까지 연쇄 교체
<code>aws_subnet</code> · <code>map_public_ip_on_launch</code> , <code>tags</code>	~	

리소스 · 속성	plan	실제로 무슨 일이
<code>aws_vpc</code> · <code>cidr_block</code>	<code>-/+</code>	VPC 교체 = 안에 있던 것 전부 연쇄 교체. 실습에 쓰지 말 것
<code>aws_vpc</code> · <code>enable_dns_hostnames</code> , <code>tags</code>	<code>~</code>	
<code>aws_security_group</code> · <code>name</code> , <code>description</code>	<code>-/+</code>	⚠ ENI에 붙어 있어 삭제가 <code>DependencyViolation</code> 으로 실패할 수 있다
<code>aws_security_group</code> · <code>ingress</code> / <code>egress</code> , <code>tags</code>	<code>~</code>	규칙만 갱신
<code>aws_route_table</code> · <code>route</code>	<code>~</code>	
<code>aws_route_table_association</code> · <code>subnet_id</code>	<code>-/+</code>	교체

! 꼭 기억

`~`가 안전하다는 뜻이 아닙니다. `instance_type` 변경은 `~` 인데도 인스턴스가 멈췄다 뜁니다. 기호는 "Terraform이 리소스를 새로 만드는가"만 알려주고, 서비스 영향은 별개입니다.

교체가 불가피할 때 다운타임을 줄이는 `lifecycle { create_before_destroy = true }`, 실수로 못 지우게 막는 `prevent_destroy` 는 이름만 두고 갑니다(뒤 주차).

[함정] 13. cycle — 서로를 참조하면 그래프가 안 만들어진다

핵심: A가 B를, B가 A를 참조하면 Terraform은 시작점을 못 찾고 아무것도 못 한다.

4번의 DAG에서 A(Acyclic, 비순환) 가 조건이었습니다. 고리가 생기면 이렇게 됩니다.

```
resource "aws_security_group" "a" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.b.id] # a -> b
  }
}

resource "aws_security_group" "b" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.a.id] # b -> a ← 고리 완성
  }
}
```

Error: Cycle: aws_security_group.a, aws_security_group.b



팁

이 에러는 plan 단계에서 납니다. AWS에는 아무 일도 일어나지 않으므로 안전하게 실험해볼 수 있습니다(실습 B-7 심화).

해결은 규칙을 리소스 밖으로 빼는 것입니다. `aws_vpc_security_group_ingress_rule`이라는 별도 리소스로 분리하면 SG는 서로를 모르고, 규칙 리소스만 양쪽을 참조하므로 화살표가 한 방향으로 정리됩니다.



주의

다만 인라인 `ingress / egress` 블록과 `aws_vpc_security_group_ingress_rule` 리소스를 섞어 쓰지 마세요. 두 방식이 서로의 규칙을 지웁니다. 오늘 코드는 인라인만 씁니다.

[개념] 14. 정리 — 세 문장, 그리고 Week3로 넘기는 것

핵심: 참조가 그래프를 만들고, 그래프가 순서를 만들고, 그 결과가 state에 적힌다.

오늘을 세 문장으로 줄이면 이렇습니다.

1. 참조가 간선이다. `vpc_id = aws_vpc.main.id` 한 줄이 화살표 하나이고, 화살표를 다 그으면 순서는 적지 않아도 된다.
2. 그래프는 순서와 병렬을 동시에 정한다. 같은 레벨은 동시에, 다음 레벨은 그다음에. `destroy`는 그 화살표를 뒤집은 것.
3. state는 결과와 그래프를 함께 적는다. `attributes`는 plan의 근거이고, `dependencies`는 `destroy` 순서의 근거다.

그리고 오늘 만든 것들이 Week3의 문제를 그대로 만들어냅니다.

오늘 본 것	Week3에서 터지는 문제
<code>terraform.tfstate</code> 가 내 노트북에만 있다	팀원은 이 장부를 볼 수 없다 → 원격 backend(S3)
<code>serial</code> 이 저장마다 +1 된다	두 사람이 동시에 apply하면 <code>serial</code> 이 충돌한다 → 잠금(lock)
state는 평문 JSON이다	원격에 두려면 암호화·접근 제어가 필요하다
<code>lineage</code> 가 장부의 혈통을 보증한다	잘못된 state를 밀어 넣는 사고를 어떻게 막나

참고 문서

- [Create Resource Dependencies](#) — 암묵적/명시적 의존성. 4·6번의 출처
- [depends_on \(Meta-Argument\)](#) — 6번
- [Resource Behavior](#) — `~`와 `-/+`의 규칙. 12번
- [Terraform State](#) · [State CLI](#) — 9·10번
- [Manage Resource Drift](#) — 11번
- [terraform graph](#) — 4번
- [aws_ami \(data source\)](#) · [aws_availability_zones](#) — 7번
- [aws_instance](#) · [aws_subnet](#) · [aws_security_group](#)
- Week1 개념워크북 2·3·4·7·8·11번 — 오늘 재설명하지 않은 부분

이 문서의 AMI 필터·AZ·비용·plan 출력 예시는 **서울 리전(ap-northeast-2)**, **AWS provider 6.57.1**, **Terraform 1.15.8** 에서 실제로 조회·검증한 값입니다. 그림은 모두 직접 그린 것입니다.

다음 → 실습워크북 을 펴고 **practice/** 폴더로 이동하세요.